



An Introduction to Parallel programming with OpenMP*

Tim Mattson

Intel Corp.

Edited / shortened by Jan Hückelheim

Download tutorial materials onto your laptop:
git clone <https://github.com/tgmattso/ATPESC.git>

* The name "OpenMP" is the property of the OpenMP Architecture Review Board.

- ➔ • Introduction to OpenMP
 - Creating Threads
 - Synchronization
 - Parallel Loops
 - Data Environment
 - Memory Model
 - Irregular Parallelism and Tasks
 - Recap
 - Beyond the Common Core:
 - Worksharing Revisited
 - Synchronization Revisited: Options for Mutual exclusion
 - Memory models and point-to-point Synchronization
 - Programming your GPU with OpenMP
 - Thread Affinity and Data Locality
 - Thread Private Data

OpenMP* Overview

C\$OMP FLUSH

#pragma omp critical

#pragma omp single

C\$OMP THREADPRIVATE (/ABC/)

C\$OMP ATOMIC

CALL OMP_SET_NUM_THREADS(10)

OpenMP: An API for Writing Parallel Applications

- A set of compiler directives and library routines for parallel application programmers
- Greatly simplifies writing multi-threaded (MT) programs in Fortran, C and C++
- Also supports non-uniform memories, vectorization and GPU programming

#pragma omp parallel for private(A, B)

C\$OMP PARALLEL REDUCTION (+: A, B)

C\$OMP PARALLEL COPYIN (/blk/)

C\$OMP DO lastprivate(XX)

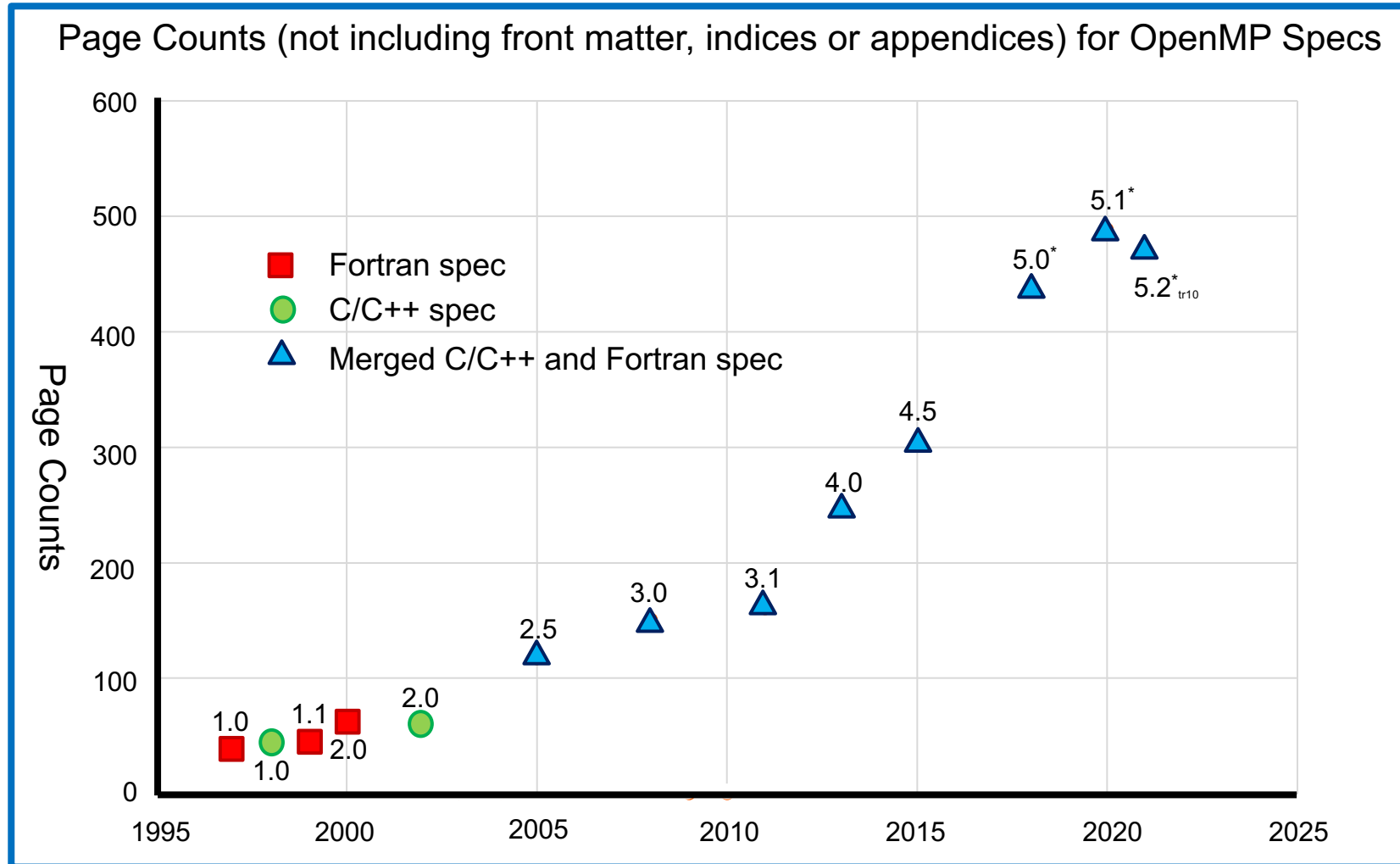
#pragma omp atomic seq_cst

Nthrds = OMP_GET_NUM_PROCS()

omp_set_lock(lck)

The Growth of Complexity in OpenMP

Our goal in 1997 ... A simple interface for application programmers



The full spec is overwhelming. We focus on the Common Core: the 21 items most people restrict themselves to

The OpenMP Common Core: Most OpenMP programs only use these 21 items

OpenMP pragma, function, or clause	Concepts
#pragma omp parallel	Parallel region, teams of threads, structured block, interleaved execution across threads.
void omp_set_thread_num() int omp_get_thread_num() int omp_get_num_threads()	Default number of threads and internal control variables. SPMD pattern: Create threads with a parallel region and split up the work using the number of threads and the thread ID.
double omp_get_wtime()	Speedup and Amdahl's law. False sharing and other performance issues.
setenv OMP_NUM_THREADS N	Setting the internal control variable for the default number of threads with an environment variable
#pragma omp barrier #pragma omp critical	Synchronization and race conditions. Revisit interleaved execution.
#pragma omp for #pragma omp parallel for	Worksharing, parallel loops, loop carried dependencies.
reduction(op:list)	Reductions of values across a team of threads.
schedule (static [,chunk]) schedule(dynamic [,chunk])	Loop schedules, loop overheads, and load balance.
shared(list), private(list), firstprivate(list)	Data environment.
default(none)	Force explicit definition of each variable's storage attribute
nowait	Disabling implied barriers on workshare constructs, the high cost of barriers, and the flush concept (but not the flush directive).
#pragma omp single	Workshare with a single thread.
#pragma omp task #pragma omp taskwait	Tasks including the data environment for tasks.

OpenMP Basic Syntax

- Most of the constructs in OpenMP are compiler directives.

C and C++	Fortran
Compiler directives	
<i>#pragma omp construct [clause [clause]...]</i>	<i>!\$OMP construct [clause [clause] ...]</i>
Example	
<i>#pragma omp parallel private(x)</i> <i>{</i> <i>}</i>	<i>!\$OMP PARALLEL PRIVATE(X)</i> <i>!\$OMP END PARALLEL</i>
Function prototypes and types:	
<i>#include <omp.h></i>	<i>use OMP_LIB</i>

- Most OpenMP constructs apply to a “structured block”.
 - Structured block: a block of one or more statements with one point of entry at the top and one point of exit at the bottom.
 - It’s OK to have an exit() within the structured block.

Exercise, Part A: Hello World

Verify that your environment works

- Write a program that prints “hello world”.

git clone <https://github.com/tgmattso/OpenMPCommonCore.git>

```
#include<stdio.h>
int main()
{

    printf(" hello ");
    printf(" world \n");

}
```

Download tutorial materials :

git clone <https://github.com/tgmattso/ATPESC.git>

Exercise, Part B: Hello World

Verify that your OpenMP environment works

- Write a multithreaded program that prints “hello world”.

```
git clone https://github.com/tgmattso/OpenMP_Common_Core.git
```

```
#include <omp.h>
#include <stdio.h>
int main()
{
    #pragma omp parallel
    {

        printf(" hello ");
        printf(" world \n");

    }
}
```

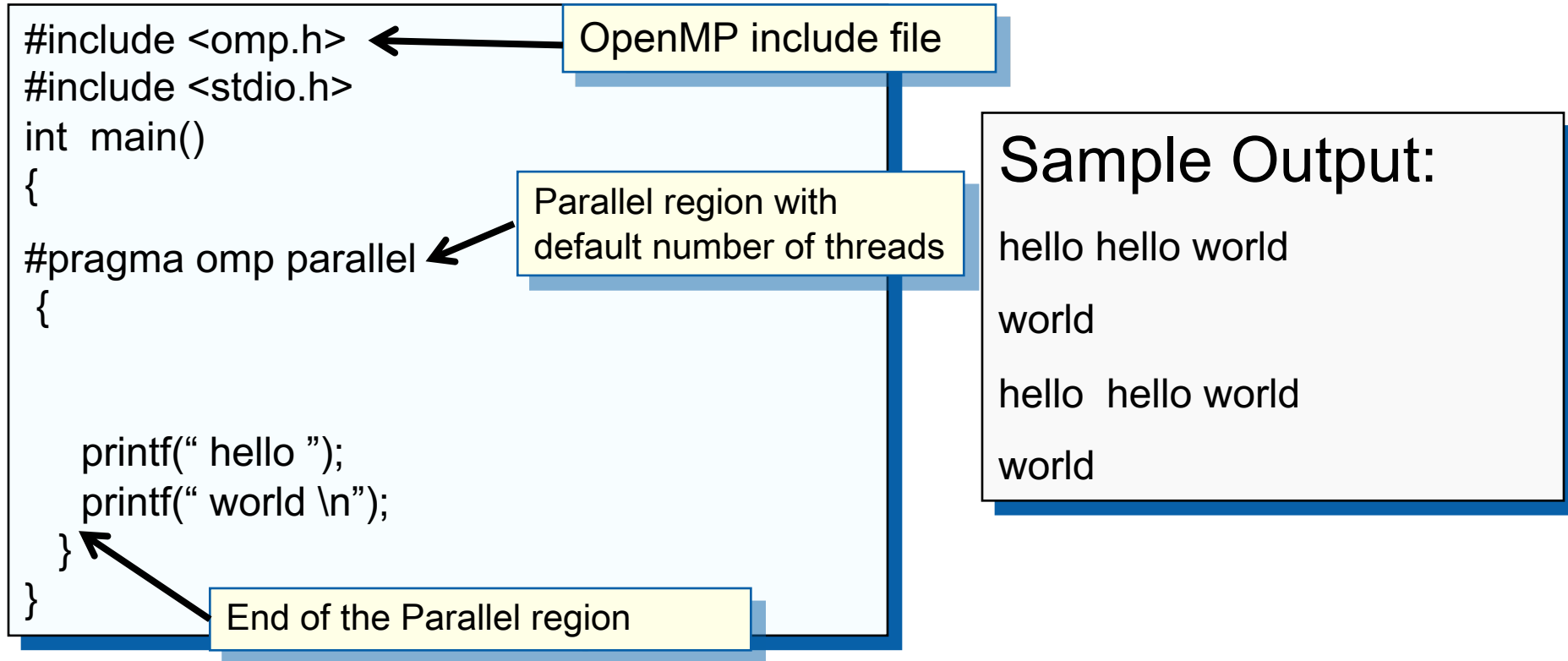
Switches for compiling and linking

gcc -fopenmp	Gnu (Linux, OSX)
cc -qopenmp	Intel (Linux@NERSC)
icl /Qopenmp	Intel (windows)
icc -fopenmp	Intel (Linux, OSX)

Solution

A Multi-Threaded “Hello World” Program

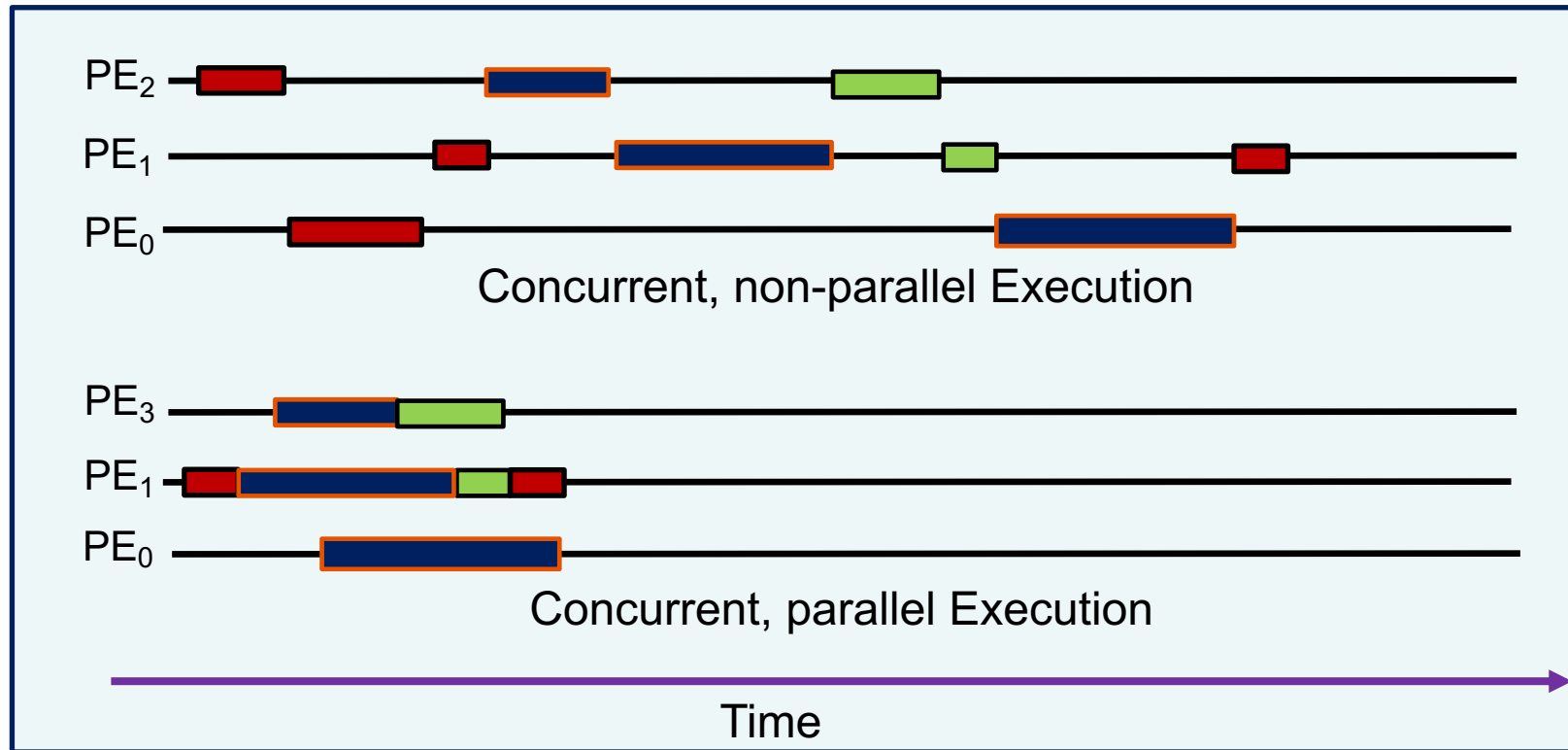
- Write a multithreaded program where each thread prints “hello world”.



The statements are interleaved based on how the operating schedules the threads

Concurrency vs. Parallelism

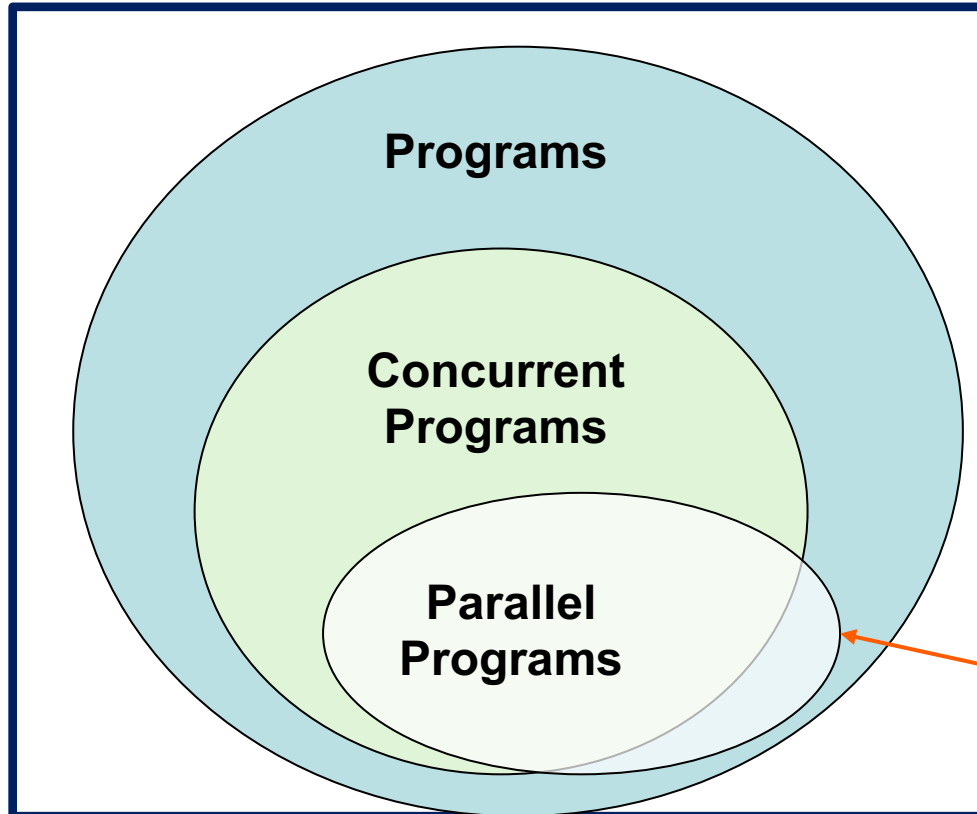
- Two important definitions:
 - Concurrency: A condition of a system in which multiple tasks are active and unordered. If **scheduled fairly**, they can be described as logically making **forward progress** at the same time.
 - Parallelism: A condition of a system in which multiple tasks are actually making **forward progress** at the same time.



PE = Processing Element

Concurrency vs. Parallelism

- Two important definitions:
 - Concurrency: A condition of a system in which multiple tasks are active and unordered. If **scheduled fairly**, they can be described as **logically** making **forward progress** at the same time.
 - Parallelism: A condition of a system in which multiple tasks are **actually** making **forward progress** at the same time.



In most cases, parallel programs exploit concurrency in a problem to run tasks on multiple processing elements

We use Parallelism to:

- Do more work in less time
- Work with larger problems

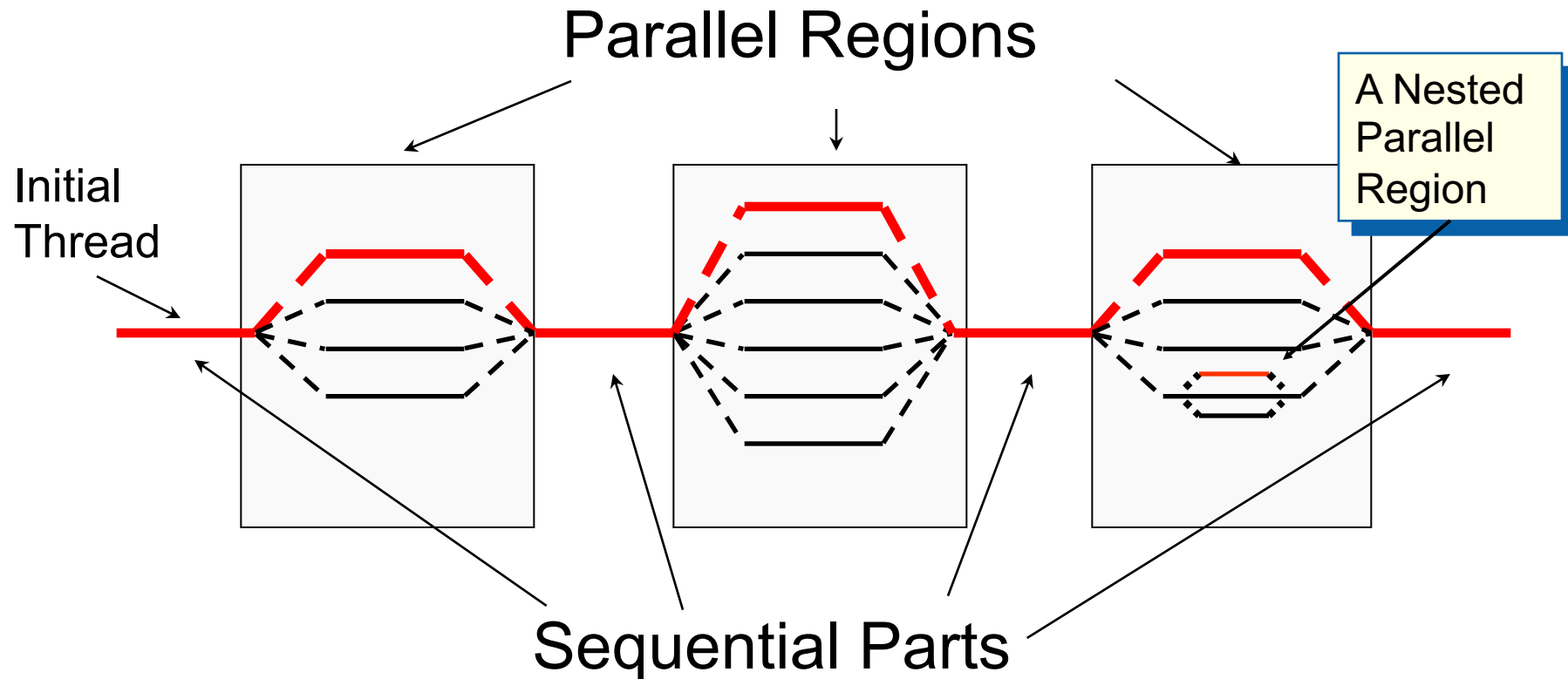
If tasks execute in “lock step” they are not concurrent, but they are still parallel.
Example ... a SIMD unit.

- Introduction to OpenMP
- ➔ • Creating Threads
- Synchronization
- Parallel Loops
- Data Environment
- Memory Model
- Irregular Parallelism and Tasks
- Recap
- Beyond the Common Core:
 - Worksharing Revisited
 - Synchronization Revisited: Options for Mutual exclusion
 - Memory models and point-to-point Synchronization
 - Programming your GPU with OpenMP
 - Thread Affinity and Data Locality
 - Thread Private Data

OpenMP Execution model:

Fork-Join Parallelism:

- ◆ Initial thread spawns a team of threads as needed.
- ◆ Parallelism added incrementally until performance goals are met, i.e., the sequential program evolves into a parallel program.



Thread Creation: Parallel Regions

- You create threads in OpenMP* with the parallel construct.
- For example, to create a 4 thread Parallel region:

Each thread executes a copy of the code within the structured block

```
double A[1000];  
omp_set_num_threads(4);  
#pragma omp parallel  
{  
    int ID = omp_get_thread_num();  
    pooh(ID,A);  
}
```

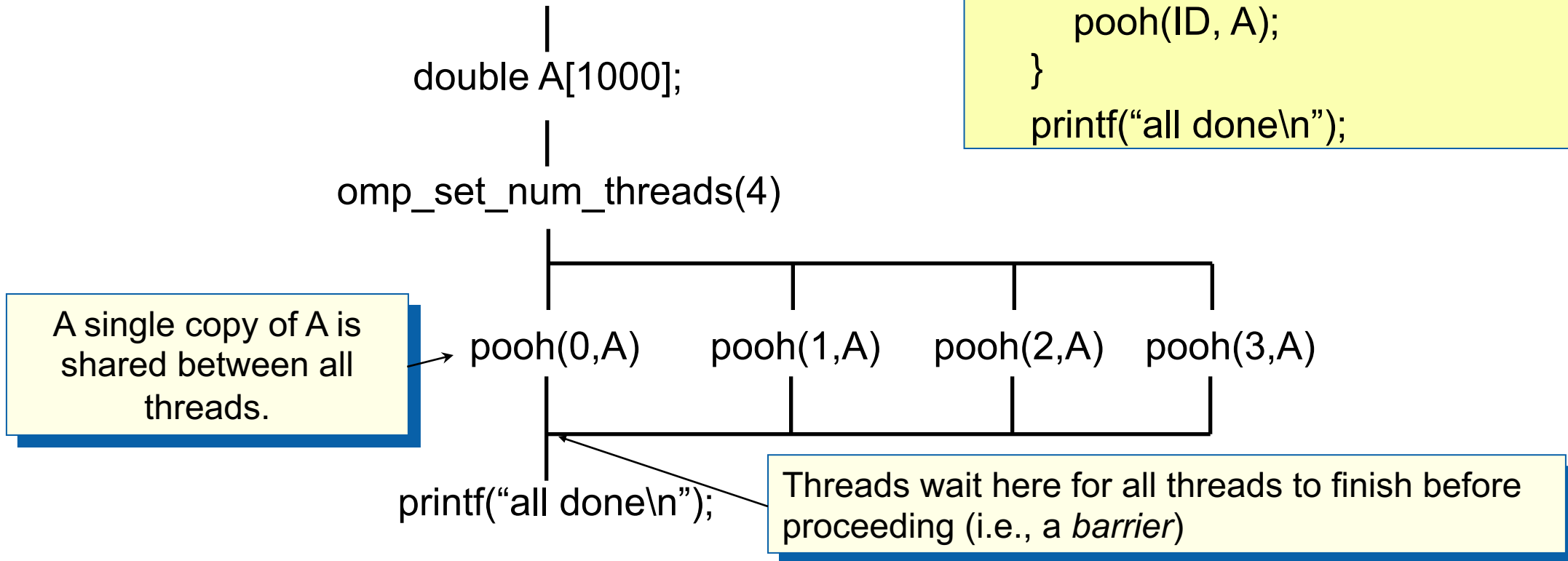
Runtime function to request a certain number of threads

Runtime function returning a thread ID

- Each thread calls pooh(ID,A) for ID = 0 to 3

Thread Creation: Parallel Regions Example

- Each thread executes the same code redundantly.



```
double A[1000];
omp_set_num_threads(4);
#pragma omp parallel
{
    int ID = omp_get_thread_num();
    pooh(ID, A);
}
printf("all done\n");
```

Thread creation: How many threads did you actually get?

- Request a number of threads with `omp_set_num_threads()`
- The number requested may not be the number you actually get.
 - An implementation may silently give you fewer threads than you requested.
 - Once a team of threads has launched, it will not be reduced.

Each thread executes a copy of the code within the structured block

```
double A[1000];
omp_set_num_threads(4);
#pragma omp parallel
{
    int ID    = omp_get_thread_num();
    int nthrds = omp_get_num_threads();
    pooh(ID,A);
}
```

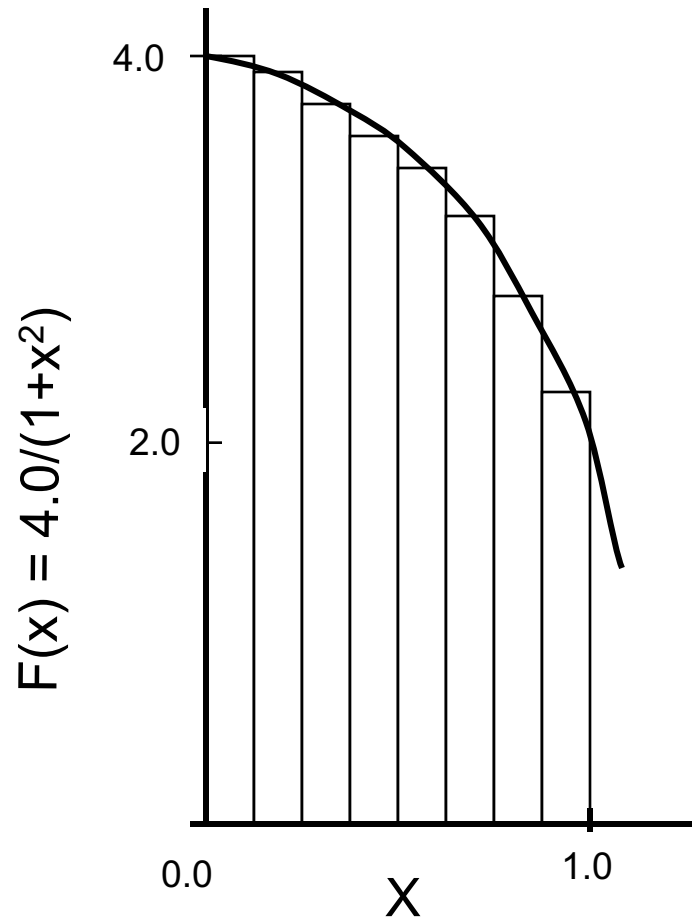
Runtime function to request a certain number of threads

Runtime function to return actual number of threads in the team

- Each thread calls `pooh(ID,A)` for `ID = 0` to `nthrds-1`

An Interesting Problem to Play With

Numerical Integration



Mathematically, we know that:

$$\int_0^1 \frac{4.0}{(1+x^2)} dx = \pi$$

We can approximate the integral as a sum of rectangles:

$$\sum_{i=0}^N F(x_i) \Delta x = \Delta x \sum_{i=0}^N F(x_i) \approx \pi$$

Where each rectangle has width Δx and height $F(x_i)$ at the middle of interval i .

Serial PI Program

```
static long num_steps = 100000;
double step;
int main ()
{
    int i;    double x, pi, sum = 0.0;

    step = 1.0/(double) num_steps;

    for (i=0;i< num_steps; i++){
        x = (i+0.5)*step;
        sum = sum + 4.0/(1.0+x*x);
    }
    pi = step * sum;
}
```

Serial PI Program

```
#include <omp.h>
static long num_steps = 100000;
double step;
int main ()
{
    int i;    double x, pi, sum = 0.0;

    step = 1.0/(double) num_steps;
double tdata = omp_get_wtime();
    for (i=0;i< num_steps; i++){
        x = (i+0.5)*step;
        sum = sum + 4.0/(1.0+x*x);
    }
    pi = step * sum;
tdata = omp_get_wtime() - tdata;
    printf(" pi = %f in %f secs\n",pi, tdata);
}
```

The library routine `get_omp_wtime()` is used to find the elapsed “wall time” for blocks of code

Exercise: the Parallel Pi Program

- Create a parallel version of the pi program using a parallel construct:
 `#pragma omp parallel`
- Pay close attention to shared versus private variables.
- In addition to a parallel construct, you will need the runtime library routines

– `int omp_get_num_threads();`

Number of threads in the team

– `int omp_get_thread_num();`

Thread ID or rank

– `double omp_get_wtime();`

– `omp_set_num_threads();`

Time in seconds since a fixed point in the past

Request a number of threads in the team

Hints: the Parallel Pi Program

- Use a parallel construct:

```
#pragma omp parallel
```

- The challenge is to:
 - divide loop iterations between threads (use the thread ID and the number of threads).
 - Create an accumulator for each thread to hold partial sums that you can later combine to generate the global sum.
- In addition to a parallel construct, you will need the runtime library routines
 - `int omp_set_num_threads();`
 - `int omp_get_num_threads();`
 - `int omp_get_thread_num();`
 - `double omp_get_wtime();`

Example: A simple SPMD pi program

```
#include <omp.h>
static long num_steps = 100000;    double step;
#define NUM_THREADS 2
void main ()
{  int i, nthreads; double pi, sum[NUM_THREADS];
  step = 1.0/(double) num_steps;
  omp_set_num_threads(NUM_THREADS);
  #pragma omp parallel
  {
    int i, id, nthrds;
    double x;
    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    if (id == 0) nthreads = nthrds;
    for (i=id, sum[id]=0.0; i< num_steps; i=i+nthrds) {
      x = (i+0.5)*step;
      sum[id] += 4.0/(1.0+x*x);
    }
  }
  for(i=0, pi=0.0; i<nthreads; i++) pi += sum[i] * step;
}
```

Promote scalar to an array dimensioned by number of threads to avoid race condition.

Only one thread should copy the number of threads to the global value to make sure multiple threads writing to the same address don't conflict.

This is a common trick in SPMD programs to create a cyclic distribution of loop iterations

Results*

- Original Serial pi program with 100000000 steps ran in 1.83 seconds.

```
#include <omp.h>
static long num_steps = 100000;      double step;
#define NUM_THREADS 2
void main ()
{  int i, nthreads; double pi, sum[NUM_THREADS];
  step = 1.0/(double) num_steps;
  omp_set_num_threads(NUM_THREADS);
  #pragma omp parallel
  {
    int i, id,nthrds;
    double x;
    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    if (id == 0)  nthreads = nthrds;
    for (i=id, sum[id]=0.0;i< num_steps; i=i+nthrds) {
      x = (i+0.5)*step;
      sum[id] += 4.0/(1.0+x*x);
    }
  }
  for(i=0, pi=0.0;i<nthreads;i++)pi += sum[i] * step;
}
```

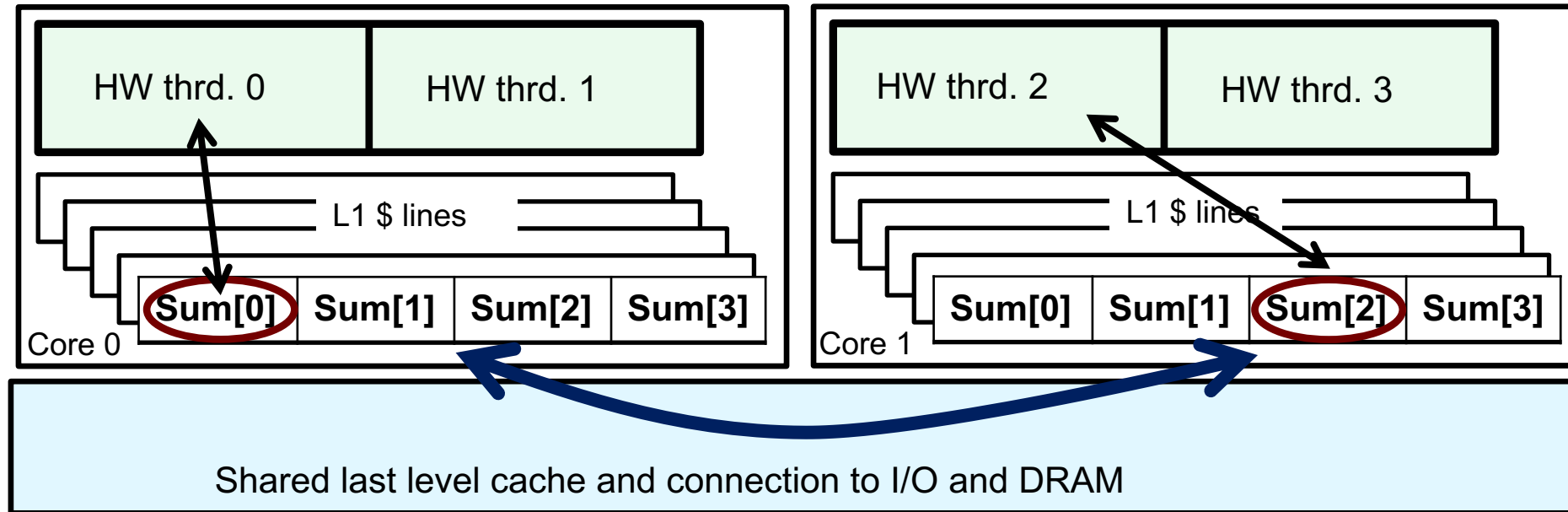
threads	1 st SPMD*
1	1.86
2	1.03
3	1.08
4	0.97

Intel compiler (icpc) with default optimization level (O2) on Apple OS X 10.7.3 with a dual core (four HW thread) Intel® Core™ i5 processor at 1.7 Ghz and 4 Gbyte DDR3 memory at 1.333 Ghz.

*SPMD: Single Program Multiple Data

Why Such Poor Scaling? False Sharing

- If independent data elements happen to sit on the same cache line, each update will cause the cache lines to “slosh back and forth” between threads ... This is called **“false sharing”**.



- If you promote scalars to an array to support creation of an SPMD program, the array elements are contiguous in memory and hence share cache lines ... Results in poor scalability.
- Solution: Pad arrays so elements you use are on distinct cache lines.

Example: Eliminate false sharing by padding the sum array

```
#include <omp.h>
static long num_steps = 100000;      double step;
#define NUM_THREADS 2
#define PAD 8      // assume 64 byte L1 cache line size
void main ()
{  int i, nthreads; double pi, sum[NUM_THREADS][PAD] ;
  step = 1.0/(double) num_steps;
  omp_set_num_threads(NUM_THREADS);
  #pragma omp parallel
  {
    int i, id, nthrds;
    double x;
    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    if (id == 0) nthreads = nthrds;
    for (i=id, sum[id]=0.0; i< num_steps; i=i+nthrds) {
      x = (i+0.5)*step;
      sum[id][0] += 4.0/(1.0+x*x);
    }
  }
  for(i=0, pi=0.0; i<nthreads; i++) pi += sum[i][0] * step;
}
```

Pad the array so each
sum value is in a
different cache line



Results*: PI Program, Padded Accumulator

- Original Serial pi program with 100000000 steps ran in 1.83 seconds.

```
#include <omp.h>
static long num_steps = 100000;      double step;
#define NUM_THREADS 2
#define PAD 8      // assume 64 byte L1 cache line size
void main ()
{  int i, nthreads; double pi, sum[NUM_THREADS][PAD] ;
  step = 1.0/(double) num_steps;
  omp_set_num_threads(NUM_THREADS);
  #pragma omp parallel
  {
    int i, id,nthrds;
    double x;
    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    if (id == 0)  nthreads = nthrds;
    for (i=id, sum[id]=0.0;i< num_steps; i=i+nthrds) {
      x = (i+0.5)*step;
      sum[id][0] += 4.0/(1.0+x*x);
    }
  }
  for(i=0, pi=0.0;i<nthreads;i++)pi += sum[i][0] * step;
}
```

threads	1 st SPMD	1 st SPMD padded
1	1.86	1.86
2	1.03	1.01
3	1.08	0.69
4	0.97	0.53

*Intel compiler (icpc) with default optimization level (O2) on Apple OS X 10.7.3 with a dual core (four HW thread) Intel® Core™ i5 processor at 1.7 Ghz and 4 Gbyte DDR3 memory at 1.333 Ghz.

- Introduction to OpenMP
- Creating Threads
-  • Synchronization
- Parallel Loops
- Data Environment
- Memory Model
- Irregular Parallelism and Tasks
- Recap
- Beyond the Common Core:
 - Worksharing Revisited
 - Synchronization Revisited: Options for Mutual exclusion
 - Memory models and point-to-point Synchronization
 - Programming your GPU with OpenMP
 - Thread Affinity and Data Locality
 - Thread Private Data

Synchronization

Synchronization is used to impose order constraints and to protect access to shared data

- High level synchronization included in the common core:
 - critical
 - barrier
- Other, more advanced, synchronization operations:
 - atomic
 - ordered
 - flush
 - locks (both simple and nested)

Synchronization: critical

- Mutual exclusion: Only one thread at a time can enter a **critical** region.

Threads wait their turn
– only one thread at a
time calls consume()

```
float res;  
  
#pragma omp parallel  
{  float B;  int i, id, nthrds;  
    id = omp_get_thread_num();  
    nthrds = omp_get_num_threads();  
    B = big_SPMD_job(id, nthrds);  
    #pragma omp critical  
        res += consume (B);  
}
```

Synchronization: barrier

- Barrier: a point in a program all threads must reach before any threads are allowed to proceed.
- It is a “stand alone” pragma meaning it is not associated with user code ... it is an executable statement.

```
double Arr[8], Brr[8];      int numthrds;

omp_set_num_threads(8)

#pragma omp parallel
{  int id, nthrds;

    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    if (id==0) numthrds = nthrds;

    Arr[id] = big_ugly_calc(id, nthrds);

    #pragma omp barrier
    Brr[id] = really_big_and_ugly(id, nthrds, Arr);
}
```

Threads wait until all
threads hit the barrier.
Then they can go on.



PI Program with False Sharing

- Original Serial pi program with 100000000 steps ran in 1.83 seconds.

```
#include <omp.h>
static long num_steps = 100000;      double step;
#define NUM_THREADS 2
void main ()
{  int i, nthreads; double pi, sum[NUM_THREADS];
  step = 1.0/(double) num_steps;
  omp_set_num_threads(NUM_THREADS);
  #pragma omp parallel
  {
    int i, id, nthrds;
    double x;
    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    if (id == 0)  nthreads = nthrds;
    for (i=id, sum[id]=0.0; i< num_steps; i=i+nthrds) {
      x = (i+0.5)*step;
      sum[id] += 4.0/(1.0+x*x);
    }
  }
  for(i=0, pi=0.0; i<nthreads; i++) pi += sum[i] * step;
}
```

Recall that promoting sum to an array made the coding easy, but led to false sharing and poor performance.

threads	1 st SPMD
1	1.86
2	1.03
3	1.08
4	0.97

*Intel compiler (icpc) with no optimization on Apple OS X 10.7.3 with a dual core (four HW thread) Intel® Core™ i5 processor at 1.7 Ghz and 4 Gbyte DDR3 memory at 1.333 Ghz.

Example: Using a critical section to remove impact of false sharing

```
#include <omp.h>
static long num_steps = 100000;      double step;
#define NUM_THREADS 2
void main ()
{ int nthreads; double pi=0.0;      step = 1.0/(double) num_steps;
  omp_set_num_threads(NUM_THREADS);
  #pragma omp parallel
  {
    int i, id, nthrds;  double x, sum;
    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    if (id == 0) nthreads = nthrds;
    for (i=id, sum=0.0; i< num_steps; i=i+nthrds) {
      x = (i+0.5)*step;
      sum += 4.0/(1.0+x*x);
    }
    #pragma omp critical
    pi += sum * step;
  }
}
```

Create a scalar local to each thread to accumulate partial sums.

No array, so no false sharing.

Sum goes “out of scope” beyond the parallel region ... so you must sum it in here. Must protect summation into pi in a critical region so updates don’t conflict

Results*: pi program critical section

- Original Serial pi program with 100000000 steps ran in 1.83 seconds.

```
#include <omp.h>
static long num_steps = 100000;      double step;
#define NUM_THREADS 2
void main ()
{ int nthreads; double pi=0.0;      step = 1.0/(double) num_steps;
  omp_set_num_threads(NUM_THREADS);
  #pragma omp parallel
  {
    int i, id, nthrds;  double x, sum;
    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();
    if (id == 0) nthreads = nthrds;
    for (i=id, sum=0.0; i< num_steps; i=i+nthrds) {
      x = (i+0.5)*step;
      sum += 4.0/(1.0+x*x);
    }
    #pragma omp critical
      pi += sum * step;
  }
}
```

threads	1 st SPMD	1 st SPMD padded	SPMD critical
1	1.86	1.86	1.87
2	1.03	1.01	1.00
3	1.08	0.69	0.68
4	0.97	0.53	0.53

*Intel compiler (icpc) with no optimization on Apple OS X 10.7.3 with a dual core (four HW thread) Intel® Core™ i5 processor at 1.7 Ghz and 4 Gbyte DDR3 memory at 1.333 Ghz.

- Introduction to OpenMP
- Creating Threads
- Synchronization
-  • Parallel Loops
- Data Environment
- Memory Model
- Irregular Parallelism and Tasks
- Recap
- Beyond the Common Core:
 - Worksharing Revisited
 - Synchronization Revisited: Options for Mutual exclusion
 - Memory models and point-to-point Synchronization
 - Programming your GPU with OpenMP
 - Thread Affinity and Data Locality
 - Thread Private Data

The Loop Worksharing Construct

- The loop worksharing construct splits up loop iterations among the threads in a team

```
#pragma omp parallel
{
  #pragma omp for
  for (I=0; I<N; I++){
    NEAT_STUFF(I);
  }
}
```

Loop construct name:

- C/C++: for
- Fortran: do

The loop control index I is made
“private” to each thread by default.

Threads wait here until all
threads are finished with the
parallel loop before any proceed
past the end of the loop

Loop Worksharing Construct

A motivating example

Sequential code

```
for(i=0;i<N;i++) { a[i] = a[i] + b[i];}
```

OpenMP parallel region
(SPMD Pattern)

```
#pragma omp parallel  
{  
    int id, i, Nthrds, istart, iend;  
    id = omp_get_thread_num();  
    Nthrds = omp_get_num_threads();  
    istart = id * N / Nthrds;  
    iend = (id+1) * (N / Nthrds)-1;  
    if (id == Nthrds-1) iend = N;  
    for(i=istart;i<iend;i++) { a[i] = a[i] + b[i];}  
}
```

OpenMP parallel region and
a worksharing for construct

```
#pragma omp parallel  
#pragma omp for  
    for(i=0;i<N;i++) { a[i] = a[i] + b[i];}
```

Loop Worksharing Constructs: The schedule clause

- The schedule clause affects how loop iterations are mapped onto threads
 - **schedule(static [,chunk])**
 - Deal-out blocks of iterations of size “chunk” to each thread.
 - **schedule(dynamic[,chunk])**
 - Each thread grabs “chunk” iterations off a queue until all iterations have been handled.
- Example:
 - #pragma omp for schedule(dynamic, 10)

Schedule Clause	When To Use
STATIC	Pre-determined and predictable by the programmer
DYNAMIC	Unpredictable, highly variable work per iteration

Least work at runtime :
scheduling done at
compile-time

Most work at runtime :
complex scheduling
logic used at run-time

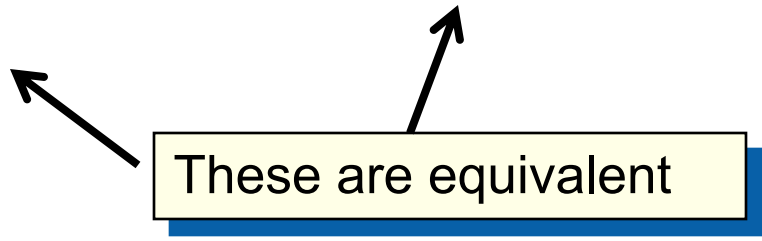
Combined Parallel/Worksharing Construct

- OpenMP shortcut: Put the “parallel” and the worksharing directive on the same line

```
double res[MAX]; int i;  
#pragma omp parallel  
{  
    #pragma omp for  
    for (i=0;i< MAX; i++) {  
        res[i] = huge();  
    }  
}
```

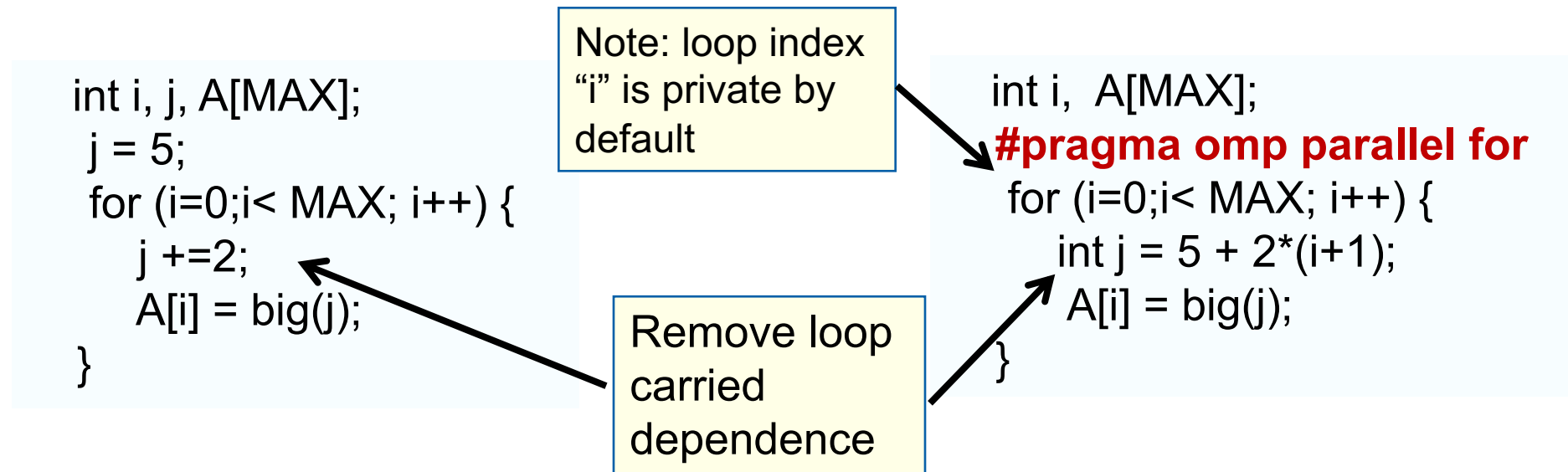
```
double res[MAX]; int i;  
#pragma omp parallel for  
    for (i=0;i< MAX; i++) {  
        res[i] = huge();  
    }
```

These are equivalent



Working with loops

- Basic approach
 - Find compute intensive loops
 - Make the loop iterations independent ... So they can safely execute in any order without loop-carried dependencies
 - Place the appropriate OpenMP directive and test



Reduction

- How do we handle this case?

```
double ave=0.0, A[MAX];  
int i;  
for (i=0;i< MAX; i++) {  
    ave + = A[i];  
}  
ave = ave/MAX;
```

- We are combining values into a single accumulation variable (ave) ... there is a true dependence between loop iterations that can't be trivially removed.
- This is a very common situation ... it is called a “reduction”.
- Support for reduction operations is included in most parallel programming environments.

Reduction

- OpenMP reduction clause:
reduction (op : list)
- Inside a parallel or a work-sharing construct:
 - A local copy of each list variable is made and initialized depending on the “op” (e.g. 0 for “+”).
 - Updates occur on the local copy.
 - Local copies are reduced into a single value and combined with the original global value.
- The variables in “list” must be shared in the enclosing parallel region.

```
double ave=0.0, A[MAX];  int i;  
#pragma omp parallel for reduction (+:ave)  
  for (i=0;i< MAX; i++) {  
    ave + = A[i];  
  }  
ave = ave/MAX;
```

OpenMP: Reduction operands/initial-values

- Many different associative operands can be used with reduction:
- Initial values are the ones that make sense mathematically.

Operator	Initial value
+	0
*	1
-	0
min	Largest pos. number
max	Most neg. number

C/C++ only	
Operator	Initial value
&	~0
 	0
^	0
&&	1
 	0

Fortran Only	
Operator	Initial value
.AND.	.true.
.OR.	.false.
.NEQV.	.false.
.IEOR.	0
.IOR.	0
.IAND.	All bits on
.EQV.	.true.

OpenMP includes user defined reductions and array-sections as reduction variables (we just don't cover those topics here)

Example: PI with a loop and a reduction

```
#include <omp.h>
static long num_steps = 100000;      double step;
void main ()
{
    double pi, sum = 0.0;
    step = 1.0/(double) num_steps;

    #pragma omp parallel for reduction(+:sum)
    for (int i=0;i< num_steps; i++){
        double x = (i+0.5)*step;
        sum = sum + 4.0/(1.0+x*x);
    }
    pi = step * sum;
}
```

Using modern C style, we put declarations close to where they are used ... which lets me use the parallel for construct.

Results*: PI with a loop and a reduction

- Original Serial pi program with 100000000 steps ran in 1.83 seconds.

Example: Pi with a		threads	1 st SPMD	1 st SPMD padded	SPMD critical	PI Loop
<pre>#include <omp.h> static long num_steps = 100000000; void main () { int i; double x, pi, sum; step = 1.0/(double) num_steps; #pragma omp parallel { double x; #pragma omp for reduction(+:sum) for (i=0; i< num_steps; i++){ x = (i+0.5)*step; sum = sum + 4.0/(1.0+x*x); } pi = step * sum; } }</pre>		1	1.86	1.86	1.87	1.91
		2	1.03	1.01	1.00	1.02
		3	1.08	0.69	0.68	0.80
		4	0.97	0.53	0.53	0.68

*Intel compiler (icpc) with no optimization on Apple OS X 10.7.3 with a dual core (four HW thread) Intel® Core™ i5 processor at 1.7 Ghz and 4 Gbyte DDR3 memory at 1.333 Ghz.

The nowait clause

- Barriers are really expensive. You need to understand when they are implied and how to skip them when it's safe to do so.

```
double A[big], B[big], C[big];
```

```
#pragma omp parallel  
{
```

```
    int id=omp_get_thread_num();
```

```
    A[id] = big_calc1(id);
```

```
#pragma omp barrier
```

```
#pragma omp for
```

```
    for(i=0;i<N;i++){C[i]=big_calc3(i,A);} 
```

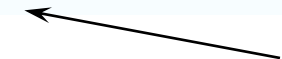
```
#pragma omp for nowait
```

```
    for(i=0;i<N;i++){ B[i]=big_calc2(C, i); } 
```

```
    A[id] = big_calc4(id);
```

```
}
```

implicit barrier at the end of a for
worksharing construct

implicit barrier at the end
of a parallel region 

no implicit barrier
due to nowait

- Introduction to OpenMP
- Creating Threads
- Synchronization
- Parallel Loops
-  • Data Environment
- Memory Model
- Irregular Parallelism and Tasks
- Recap
- Beyond the Common Core:
 - Worksharing Revisited
 - Synchronization Revisited: Options for Mutual exclusion
 - Memory models and point-to-point Synchronization
 - Programming your GPU with OpenMP
 - Thread Affinity and Data Locality
 - Thread Private Data

Data Environment:

Default storage attributes

- Shared memory programming model:
 - Most variables are shared by default
- Global variables are SHARED among threads
 - Fortran: COMMON blocks, SAVE variables, MODULE variables
 - C: File scope variables, static
 - Both: dynamically allocated memory (ALLOCATE, malloc, new)
- But not everything is shared...
 - Stack variables in subprograms(Fortran) or functions(C) called from parallel regions are PRIVATE
 - Automatic variables within a statement block are PRIVATE.

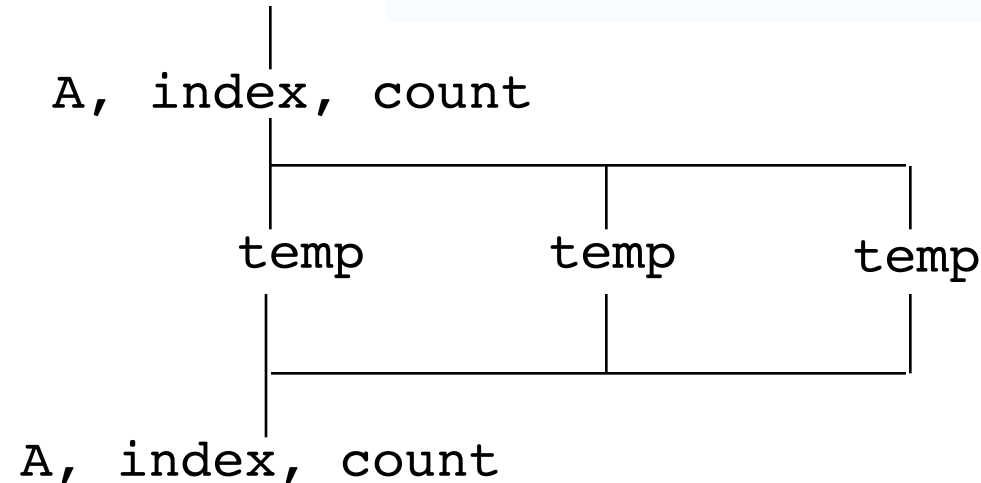
Data Sharing: Examples

```
double A[10];  
int main() {  
    int index[10];  
    #pragma omp parallel  
        work(index);  
    printf("%d\n", index[0]);  
}
```

A, index and count are shared by all threads.

temp is local to each thread

```
extern double A[10];  
void work(int *index) {  
    double temp[10];  
    static int count;  
    ...  
}
```



Data Sharing: Changing storage attributes

- One can selectively change storage attributes for constructs using the following clauses* (note: *list* is a comma-separated list of variables)
 - shared(list)
 - private(list)
 - firstprivate(list)
- These can be used on parallel and for constructs ... other than shared which can only be used on a parallel construct
- Force the programmer to explicitly define storage attributes
 - default (none)

default() can only be used
on parallel constructs

Data Sharing: Private clause

- `private(var)` creates a new local copy of `var` for each thread.

```
int N = 1000;  
extern void init_arrays(int N, double *A, double *B, double *C);
```

```
void example () {  
    int i, j;  
    double A[N][N], B[N][N], C[N][N];  
    init_arrays(N, *A, *B, *C);  
  
    #pragma omp parallel for private(j)  
    for (i = 0; i < 1000; i++)  
        for( j = 0; j<1000; j++)  
            C[i][j] = A[i][j] + B[i][j];  
}
```

OpenMP makes the loop control index on the parallel loop (i) private by default ... but not for the second loop (j)

Data Sharing: Private clause

- `private(var)` creates a new local copy of `var` for each thread.
 - The value of the private copies is uninitialized
 - The value of the original variable is unchanged after the region

```
void wrong() {  
    int tmp = 0;  
    #pragma omp parallel for private(tmp)  
    for (int j = 0; j < 1000; ++j)  
        tmp += j;  
    printf("%d\n", tmp);  
}
```

When you need to refer to the variable `tmp` that exists prior to the construct, we call it the **original variable**.

tmp is 0 here

tmp was not initialized

Data Sharing: Private and the original variable

- The original variable's value is unspecified if it is referenced outside of the construct
 - Implementations may reference the original variable or a copy a dangerous programming practice!
 - For example, consider what would happen if the compiler inlined work()?

```
int tmp;  
void danger() {  
    tmp = 0;  
    #pragma omp parallel private(tmp)  
        work();  
    printf("%d\n", tmp);  
}
```



tmp has unspecified value

```
extern int tmp;  
void work() {  
    tmp = 5;  
}
```

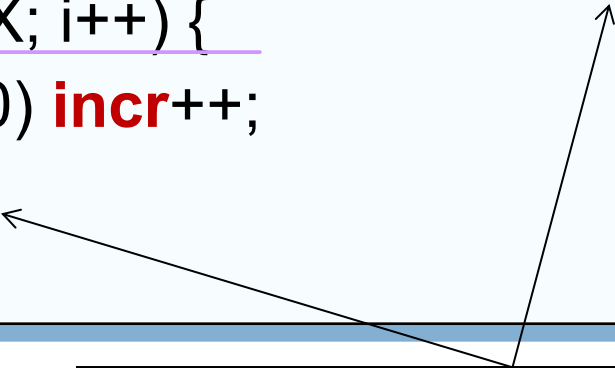


unspecified which
copy of tmp

Firstprivate clause

- Variables initialized from a shared variable
- C++ objects are copy-constructed

```
incr = 0;  
#pragma omp parallel for firstprivate(incr)  
for (i = 0; i <= MAX; i++) {  
    if ((i%2)==0) incr++;  
    A[i] = incr;  
}
```



Each thread gets its own copy of
incr with an initial value of 0

Data sharing:

A data environment test

- Consider this example of PRIVATE and FIRSTPRIVATE

```
variables: A = 1, B = 1, C = 1  
#pragma omp parallel private(B) firstprivate(C)
```

- Are A,B,C private to each thread or shared inside the parallel region?
- What are their initial values inside and values after the parallel region?

Inside this parallel region ...

- “A” is shared by all threads; equals 1
- “B” and “C” are private to each thread.
 - B’s initial value is undefined
 - C’s initial value equals 1

Following the parallel region ...

- B and C revert to their original values of 1
- A is either 1 or the value it was set to inside the parallel region

Data Sharing: Default clause

- **default(none)**: Forces you to define the storage attributes for variables that appear inside the static extent of the construct ... if you fail the compiler will complain. Good programming practice!
- You can put the default clause on parallel and parallel + workshare constructs.

The static extent is the code in the compilation unit that contains the construct.

```
#include <omp.h>
int main()
{
    int i, j=5;    double x=1.0, y=42.0;
    #pragma omp parallel for default(none) reduction(*:x)
    for (i=0; i<N; i++){
        for(j=0; j<3; j++)
            x+= foobar(i, j, y);
    }
    printf(" x is %f\n", (float)x);
}
```

The compiler would complain about j and y, which is important since you don't want j to be shared

The full OpenMP specification has other versions of the default clause, but they are not used very often so we skip them in the common core

- Introduction to OpenMP
- Creating Threads
- Synchronization
- Parallel Loops
- Data Environment
- Memory Model
-  • Irregular Parallelism and Tasks
- Recap
- Beyond the Common Core:
 - Worksharing Revisited
 - Synchronization Revisited: Options for Mutual exclusion
 - Memory models and point-to-point Synchronization
 - Programming your GPU with OpenMP
 - Thread Affinity and Data Locality
 - Thread Private Data

Irregular Parallelism

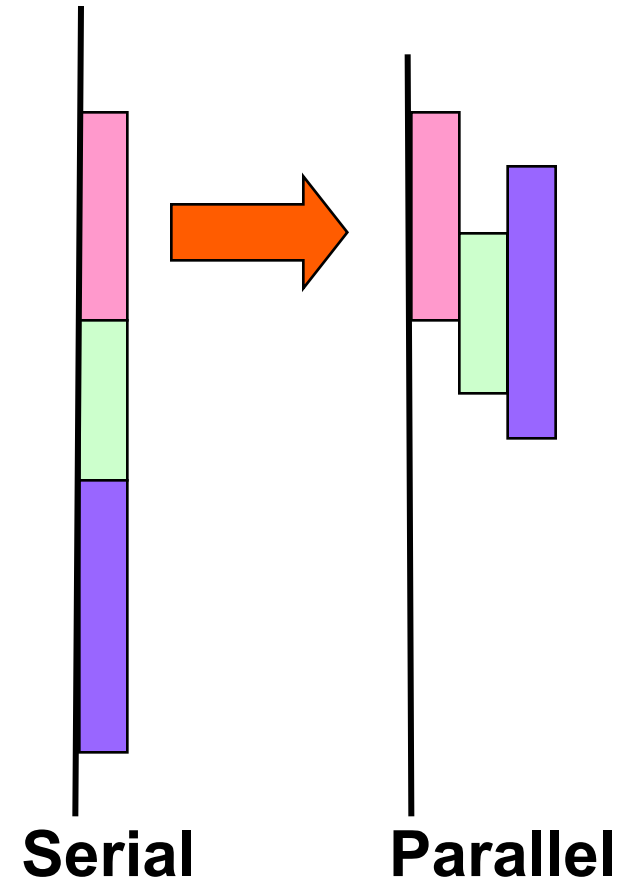
- Let's call a problem “irregular” when one or both of the following hold:
 - Data Structures are sparse
 - Control structures are not basic for-loops
- Example: Traversing Linked lists:

```
p = listhead ;  
while (p) {  
    process(p) ;  
    p=p->next;  
}
```

- Using what we've learned so far, traversing a linked list in parallel using OpenMP is difficult.

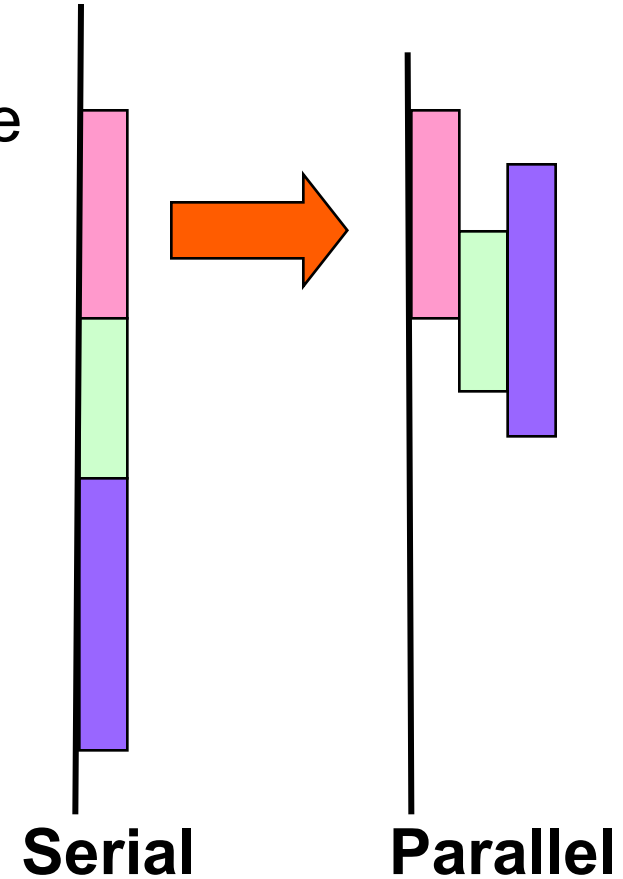
What are Tasks?

- Tasks are independent units of work
- Tasks are composed of:
 - code to execute
 - data to compute with
- Threads are assigned to perform the work of each task.
 - The thread that encounters the task construct may execute the task immediately.
 - The threads may defer execution until later



What are Tasks?

- The task construct includes a structured block of code
- Inside a parallel region, a thread encountering a task construct will package up the code block and its data for execution
- Tasks can be nested: i.e. a task may itself generate tasks.



A common Pattern is to have one thread create the tasks while the other threads wait at a barrier and execute the tasks

Single Worksharing Construct

- The **single** construct denotes a block of code that is executed by only one thread (not necessarily the primary* thread).
- A barrier is implied at the end of the single block (can remove the barrier with a *nowait* clause).

```
#pragma omp parallel
{
    do_many_things();
    #pragma omp single
        { exchange_boundaries(); }
        do_many_other_things();
}
```

*This used to be called the “master thread”. The term “master” has been deprecated in OpenMP 5.1 and replaced with the term “primary”.

Task Directive

`#pragma omp task [clauses]`
structured-block

